

10기_research팀_장서연_요약

논문 정보

- **제목:** Efficient Memory Management for Large Language Model Serving with PagedAttention
 - **저자:** Kwon et al., 2023
 - **발표자:** 이재린 (EWha EURON)
-

#01 Introduction — 연구 배경 및 분야

- **분야:** LLM Serving 시스템의 효율적인 메모리 관리
 - **목표:** KV cache 메모리 관리 최적화로 LLM serving throughput 향상
 - **응용:** GPT, PaLM 기반 프로그래밍 어시스턴트, 챗봇 등 클라우드 호스팅 서비스
 - **배경 수치:** LLM 요청 처리 비용은 기존 키워드 검색 대비 **10배 이상** → throughput 향상 = cost per request 감소가 핵심
 - **메모리 분포 (13B / A100):** Parameters 65% (26GB), KV Cache >30%, Others
 - **LLM 동작 방식:**
 - Autoregressive Transformer → token을 하나씩 순차 생성
 - 매 step마다 이전 모든 token의 KV cache 필요
 - Sequential 구조 → workload가 **memory-bound** → GPU 연산 활용도 낮음, throughput 제한
-

#01 Introduction — 기존 연구의 한계점

- **근본 원인:** 기존 시스템(FasterTransformer, Orca)은 KV cache를 연속된 (contiguous) 메모리에 저장

1. Internal & External Fragmentation

- 요청의 최대 길이(예: 2048 tokens) 기준으로 미리 메모리 할당(pre-allocation)
- 실제 출력 길이는 훨씬 짧은 경우가 많음 → **internal fragmentation**
- 요청마다 pre-allocated 크기가 달라 메모리 할당자에서 **external fragmentation**
- 예약된(reserved) 공간도 요청이 끝날 때까지 다른 요청이 사용 불가

2. Memory Sharing 불가

- Parallel sampling, beam search 등은 여러 sequence가 KV cache 일부 공유 가능
 - 하지만 각 sequence의 KV cache가 별도 contiguous space에 저장 → 공유 불가
 - 수치: 기존 시스템에서 실제 token state 사용 KV cache 비율 = **20.4% ~ 38.2%**
-

#01 Introduction — 논문의 Contributions

- 제안: OS의 virtual memory & paging에서 영감 → **PagedAttention** + LLM serving engine **vLLM**

vLLM의 기여

- (1) **Near-zero waste** in KV cache memory
- (2) **Flexible sharing** of KV cache (within & across requests)

Key Contributions

1. LLM serving의 메모리 할당 문제 식별 & 성능 영향 정량화
 2. PagedAttention 제안
 3. vLLM 설계 및 구현
 4. 다양한 시나리오 평가
-

#02 Background — Transformer & KV Cache

KV Cache (Autoregressive Generation)

- LLM이 token 생성 시 매 step마다 이전 token의 key, value vector를 **재계산하지 않고** 저장해두는 메커니즘

두 가지 생성 단계

1. **Prompt phase:** 전체 입력 prompt를 한 번에 처리 → matrix-matrix multiplication → GPU parallelism 효율적 활용
2. **Autoregressive generation phase:** 매 iteration마다 token 1개씩 생성 → matrix-vector multiplication → GPU 활용 low, memory-bound, latency 대부분 차지

Batching

- 여러 요청을 묶어 처리 → model weights 이동 overhead 분산 → throughput 향상
- **Iteration-level scheduling:** 매 iteration마다 완료된 요청 제거 + 새 요청 추가 → queueing delay 감소, padding 제거 → fine-grained batching

#03 Memory Challenges in LLM Serving

세 가지 메모리 문제

1. Large KV Cache

- 요청 수 ↑ → KV cache 크기 급격히 증가 → 메모리가 점점 더 큰 bottleneck

2. Complex Decoding Algorithms

- 기존 시스템은 parallel sampling, beam search에서 sharing 기회를 전혀 활용하지 못함

3. Scheduling for Unknown Input & Output Lengths

- 출력 길이를 사전에 알 수 없음 → 메모리 부족 시 일부 요청의 KV cache를 삭제하거나 swap out하는 scheduling 결정 필요

#04 Method — PagedAttention

핵심 아이디어

- OS의 virtual memory & paging → KV cache 관리에 적용
- **blocks = pages, tokens = bytes, requests = processes**

동작 방식

- 각 sequence의 KV cache를 고정 크기 KV block으로 분할
- 각 block은 고정된 수(block size B)의 token에 대한 key, value vector 저장
- KV block들은 **non-contiguous physical memory**에 저장 가능

Attention 계산 (Eq. 4)

- 기존 attention을 **block-wise computation**으로 변환
- PagedAttention kernel이 block table을 참조하여 각 KV block을 개별적으로 fetch하여 연산

효과

- **Internal fragmentation**: 작은 block + on-demand 할당으로 완화
- **External fragmentation**: 모든 block이 동일한 크기 → 제거
- **Memory sharing**: block 단위로 가능

#04 Method — KV Cache Manager & Block

KV Cache Manager 구조

- **Logical KV blocks** → **Physical KV blocks** 매핑 (OS virtual memory와 동일)
- **Block table**: 각 요청의 logical ↔ physical block 매핑 + filled position 기록
- Logical-physical 분리 → 필요할 때만 physical block 동적 할당 → 메모리 낭비 대부분 제거

Decoding 예시

1. **Prefill step**: logical block 0(4개 token) → physical block 7, logical block 1(3개 token + 1 reserved) → physical block 10에 매핑
2. **첫 번째 generation step**: PagedAttention으로 새 token 생성 → 남은 reserved slot에 KV cache 저장
3. **두 번째 generation step**: 마지막 logical block이 가득 차면 → 새 physical block (block 3) 할당 후 매핑

메모리 낭비 최소화

- 모든 block을 왼쪽부터 채우고 이전 block이 가득 찰 때만 새 block 할당
 - 요청당 메모리 낭비는 **마지막 block 하나 이내**로 제한
 - 요청 완료 시 KV block 즉시 해제 → 다른 요청이 재사용
-

#04 Method — 다양한 Decoding 시나리오 적용 (1)

1. Parallel Sampling

- 하나의 input prompt에서 여러 output 생성 (예: 프로그래밍 어시스턴트의 코드 후보 제시)
 - Prompt KV cache는 모든 sample이 동일 → **하나의 physical block에 저장 후 공유**
 - 각 physical block에 **reference count** 도입 → 몇 개의 logical block이 참조하는지 추적
 - 마지막 logical block 하나만 **copy-on-write**, 나머지 prompt block은 완전 공유
 - Long prompt일수록 메모리 절약 효과 큼
-

#04 Method — 다양한 Decoding 시나리오 적용 (2)

2. Beam Search

- 상위 k개의 가장 그럴듯한 output sequence 유지하며 디코딩
- Parallel sampling과 달리 **prompt block + 중간 block도 candidate** 간 공유 가능
- Sharing pattern이 decoding 진행에 따라 동적으로 변화 → OS의 **compound fork process tree**와 유사

3. Shared Prefix

- 사용자가 task description + 예시 입출력(system prompt)을 긴 prefix로 제공 (예: few-shot 번역)
- vLLM: LLM service provider가 **predefined shared prefix의 physical block을 미리 예약**
- 해당 prefix를 포함한 요청은 logical block을 미리 캐싱된 physical block에 직접 매핑 (마지막 block만 copy-on-write)
- Prompt phase 계산은 사용자의 task input 부분만 실행 → **prefix 계산 중복 제거**
- OS의 shared library 공유 방식과 동일 개념

4. Mixed Decoding Methods

- 서로 다른 decoding 방식을 같은 batch에서 처리 가능

#04 Method — Scheduling & Preemption

Scheduling 정책

- **FCFS (First-Come-First-Serve)** → 공정성 보장, starvation 방지
- Preemption 시: 가장 늦게 도착한 요청을 먼저 preempt

Preemption 발생 조건 및 정책

- GPU의 physical block 부족 → 일부 요청 preempt

- **All-or-nothing eviction policy:** 한 sequence의 block은 전부 evict or 전부 유지 (모든 block이 함께 접근되기 때문)
- Beam search처럼 memory sharing이 있는 sequence group은 **gang-scheduled**
→ 항상 함께 preempt/reschedule

복구 방법 2가지

- **Swapping:** evicted blocks를 CPU 메모리로 swap out
- **Recomputation:** 다시 prefill phase를 통해 KV cache 재계산

#04 Method — Distributed Execution

- 대형 모델은 single GPU 용량 초과 → **Megatron-LM 방식의 tensor model parallelism** 지원 (SPMD)
- Attention operator는 attention head 차원으로 분할, GPU 간 **all-reduce**로 중간 결과 동기화
- **단일 KV cache manager**를 centralized scheduler 내에 유지 → 각 GPU worker가 동일한 logical-physical block mapping 공유
- 각 GPU worker는 해당 attention head에 대한 KV cache 부분만 저장

#05 Experiment — 실험 설정

Models

- OPT-13B / OPT-66B / OPT-175B (GPT-3 규모) / LLaMA-13B

Hardware (Google Cloud A2, NVIDIA A100)

- 13B: A100 1개 (40GB)
- 66B: 4×A100 (160GB)
- 175B: 8×A100-80GB (640GB)

Datasets

- **ShareGPT**: 실제 ChatGPT 대화 데이터, 평균 input 161 / output 338토큰, high variance
- **Alpaca**: GPT-3.5 생성 instruction dataset, 평균 input 19 / output 58토큰 (ShareGPT 대비 8.4× 짧은 input, 5.8× 짧은 output)

Baselines

- **FasterTransformer**: latency 최적화 distributed inference engine, dynamic batching 추가 구현, Orca Max 방식 KV cache 관리
- **Orca (Max)**: 항상 최대 sequence length(2048)로 메모리 예약
- **Orca (Pow2)**: 실제 출력 길이의 최대 2배로 예약 (예: 25 → 32)
- **Orca (Oracle)**: 실제 출력 길이를 미리 안다고 가정한 upper bound (실현 불가)

Key Metric

- **Normalized latency (s/token)** = end-to-end latency / output length
- High-throughput 시스템 = 높은 request rate에서도 낮은 normalized latency 유지

#05 Experiment — Basic Sampling 결과

- **ShareGPT trace**: vLLM이 Orca (Oracle) 대비 **1.7~2.7×**, Orca (Max) 대비 **2.7~8×** higher throughput
 - **Alpaca trace**: 모든 모델에서 vLLM이 baseline 압도
 - Batched requests 수 (예시):
 - ShareGPT: Orca(Max) 7.00 / Orca(Pow2) 9.81 / Orca(Oracle) 13.62 / **vLLM 30.42**
 - Alpaca: Orca(Max) 7.00 / Orca(Pow2) 43.24 / Orca(Oracle) 72.75 / **vLLM 132.44**
 - 결론: 효율적 메모리 관리 덕분에 더 많은 요청을 동시에 batch 가능
-

#05 Experiment — Parallel Sampling & Beam Search / Shared Prefix / Chatbot

- **Parallel Sampling:** parallel size 2/4/6 모두에서 vLLM이 baseline 대비 우수, parallel size 클수록 격차 증가 (KV cache 공유 효과)
 - **Beam Search:** beam width 2/4/6에서 vLLM이 baseline 대비 큰 폭의 throughput 향상 (중간 block 공유 효과)
 - **Shared Prefix:** 미리 캐싱된 physical block 재사용으로 prompt 처리 비용 절감
 - **Chatbot:** 긴 대화 컨텍스트에서도 vLLM이 메모리 효율 덕분에 안정적 throughput 유지
-

#06 Ablation Studies — Kernel & Block Size

1. Kernel Microbenchmark

- PagedAttention은 기존 시스템이 효율적으로 지원하지 않는 memory access pattern 도입 → 전용 GPU kernel 3가지 개발
 - Fused reshape and block write
 - Fusing block read and attention
 - Fused block copy
- **결과:** FasterTransformer 대비 attention kernel latency **20~26% 증가** (overhead는 attention operator에만 국한)
- End-to-end 성능에서는 PagedAttention 덕분에 vLLM이 FasterTransformer를 크게 앞섬

2. Impact of Block Size

- Block size 너무 작음 → GPU parallelism 활용 부족
- Block size 너무 큼 → internal fragmentation 증가 + sharing 확률 감소
- **실험 결과:**
 - ShareGPT: block size **16~128** 범위에서 최적

- Alpaca: block size **16, 32**가 잘 작동 / 더 큰 size는 sequence 길이보다 block이 커져 성능 급락
- **기본값: block size 16** (GPU 활용 + fragmentation 최소화 균형점)

#06 Ablation Studies — Recomputation vs Swapping

Swapping

- Block size 작을수록 CPU-GPU 간 수많은 소규모 데이터 전송 → PCIe bandwidth 효율 저하 → overhead 큼
- Block size 클수록 효율적

Recomputation

- Block size와 무관하게 일정한 overhead 유지 (KV block을 직접 사용하지 않으므로)
- Recomputation overhead는 swapping latency의 ****최대 20%****를 넘지 않음

결론

- Block size 작을 때 → **recomputation** 유리
- Block size 클 때 → **swapping** 유리
- Block size **16~64** 중간 범위 → 두 방법 **end-to-end** 성능 유사

#07 Related Work — 기존 연구와의 비교

시스템	Fine-grained Scheduling	KV Cache 효율	Memory Sharing	Online Serving
FasterTransformer	✗	낮음 (Max 방식)	✗	○
Orca	○	낮음 (contiguous)	✗	○
FlexGen	✗	중간	✗	✗

시스템	Fine-grained Scheduling	KV Cache 효율	Memory Sharing	Online Serving
OLLA	✗	중간	✗	✗
FlashAttention	✗	중간 (계산 효율)	✗	○
vLLM	○	높음 (paged)	○	○

#08 Conclusion

핵심 Contribution 요약

1. LLM serving에서 **KV cache 메모리 낭비 문제 식별 및 정량화**
2. OS의 virtual memory & paging에서 영감을 받은 **PagedAttention 제안** → non-contiguous paged memory에서 KV cache 저장
3. **vLLM 구현** → near-zero waste in KV cache memory + flexible KV cache sharing (within/across requests)
4. FasterTransformer, Orca 대비 **2~4× throughput 향상** (모델 정확도 변화 없음)
5. 긴 sequence, 큰 모델, 복잡한 decoding algorithm일수록 개선 효과 더 크게 나타남

#09 Discussion

1. PagedAttention의 block size는 trade-off가 존재한다고 논문에서 언급했는데, 고정된 단일 block size 대신 요청의 특성(sequence 길이, decoding 알고리즘 종류 등)에 따라 block size를 **동적으로 조절**하는 방식이 가능할까요? 가능하다면 어떤 기준으로 결정해야 할까요?
2. vLLM은 preemption 발생 시 **swapping과 recomputation** 두 가지 복구 방법을 제시했는데, GPU 성능과 메모리 용량이 계속 발전하는 상황에서 미래에는 어떤 방향이 더 유효한 전략이 될까요?